

1) Abstract Data Type (ADT)

Abstract Data Type is an object in which it have set of value & set of operations.

- It is logically data structure.
- It tells which operation should be performed & which should not be performed.
- Abstract data type is a data structure type in which it implement independently.

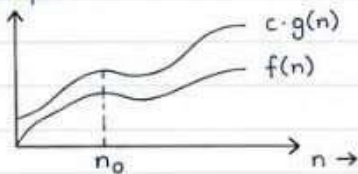
* Uses

- Help to maintain the in data engagement.
- Improved the program.
- Make the program simple.
- It uses to define the data structure.

2) Big O vs Big Ω

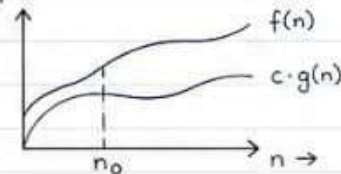
Big O

- It represent the worst case of time complexity.
- It gives upper bound of an algorithm.
- It shows the maximum growth rate of an algorithm.
- Written as $O(f(n))$.
- It used to analyze performance in worst situation.
- Graph:



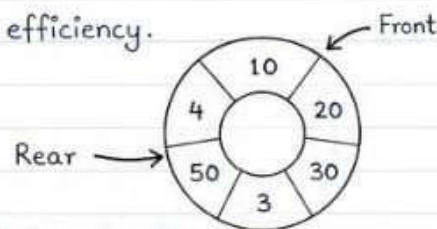
Big Ω

- It represent the best case of time complexity.
- It gives lower bound of an algorithm.
- It shows the minimum growth rate of an algorithm.
- Written as $\Omega(f(n))$.
- It used to analyze performance in best situation.
- Graph:



3) Circular Queue

- Circular Queue is similar like a simple queue but it connects the last position to the first element forming circle.
- Reduces the memory efficiency.
- Eg:



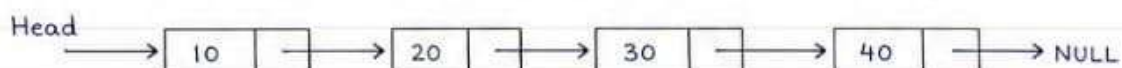
4) Example : Circular buffering in Memory

- Uses

- 1) Use in memory management system
- 2) Use in real time system.

5) Singly Linked List

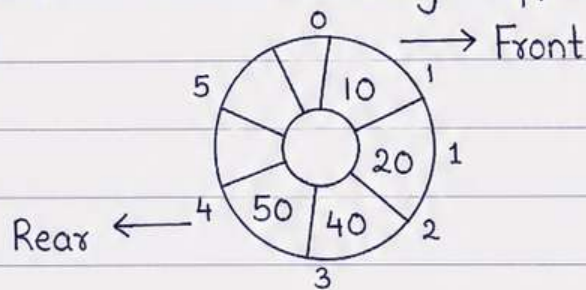
- A linked list in which each node points to the next node only.



3] Circular Queue

- Circular Queue is similar like a simple queue but it connects the last position to the first element forming circle
- Reduces the memory efficiency.

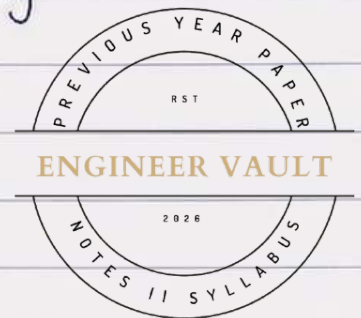
Eg:



2 Example :- Circular buffering in Memory

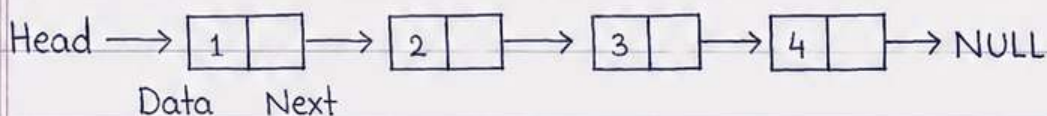
* Uses

- 1) Use in memory management system
- 2) Use in real time system.



4] Singly Linked List

- Singly Linked List means in which the each node contains
- 1) Data → the actual value
- 2) Next (Pointer) → address to the next pointer.
- It is a linear data structure.



* Uses

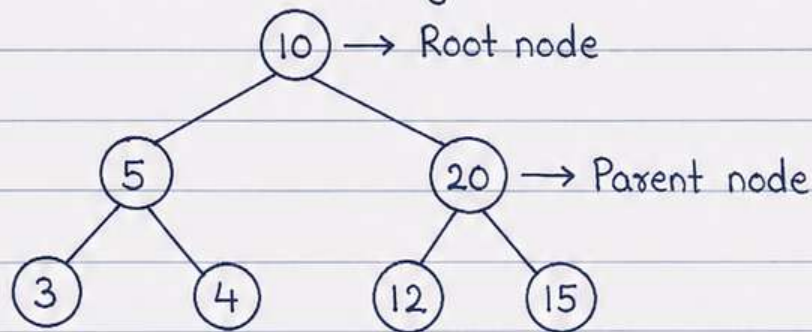
- Use to implement stack & queue
- Use in dynamic memory allocation

5] Binary Tree

- Binary Tree is a hierarchical data structure in which each node has 2 children left child & right child.
- Binary tree is non-linear data structure
- The top node of the tree is known as root node.

5] The nodes having no children are called leaf nodes.

- It is used in searching



6] Quick Sort

- It is highly divide & conquer sorting algorithm
- It works by choosing the pivot elements, partition array around pivot element & recursively sorted array
- Time Complexity
B.C, A.C, S.C = $O(n \log n)$

Merge Sort

- It is divide & conquer sorting algorithm.
- It works by divide the 2 halves recursively, sort them, & then merge them 2 halves back together.
- Time Complexity
B.C, W.C, A.C = $O(n \log n)$

7] Hashing is used to identify the specific object in form of similar object.

- It is used to convert the large or arbitrary keys (name or email) to small or fixed value called hash value using hash function.
- Data is stored using hash function.

8] Threaded Binary Tree is a binary tree data structure in which NULL left or right pointers are replaced by threads.

- These threads point to inorder predecessor & inorder successor.

Successor

- Right pointer is inorder successor
- Left pointer is inorder predecessor.

9] BFS

- Breadth First Search
- Visit node level by level
- Uses Queue for traversal
- Have more memory
- Slower for deep graphs

DFS

- Depth First Search
- Go deep first to visit node
- Uses Stack for traversal
- Have low memory
- Faster for deep graphs

10] Priority Queue

- It is used to assigned the priority.
- It removes the element with priority, not FIFO order.

* Diagram = 5 [highest], 3, 1 [lowest].
 $p=5$ $p=3$ $p=1$

* Eg:- Used in hospital emergency system.

* Uses

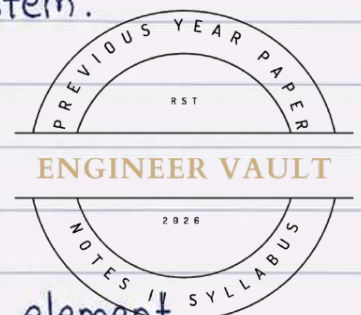
- CPU scheduling
- Print scheduling

* Types

- 1) MAX Priority Queue → Removes highest element
- 2) MIN Priority Queue → Removes lowest element.

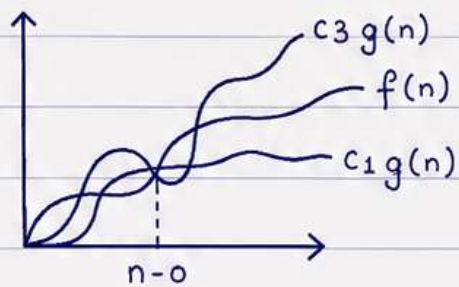
1] Big O Notation

- Represent the average case of time complexity.
- Gives upper & lower bound of an algorithm
- Shows exact growth rate of algorithm.
- It is analyze when best case & worst case are same.



11) Written as $O(f(n))$

Helps to compare the large input size of algorithm.



12) $(A+B) * (C-D)$

Size	Symbol	Stack Operators	Link Expression
1	(	(	-
2	A	C	A
3	+	C+	A
4	B	C+	AB
5	)	-	AB+
6	*	*	AB+
7	(	*C	AB+
8	C	*C	AB+C
9	-	*-	AB+C
10	D	*-	AB+CD
11	)	*	AB+CD-
12	-	-	AB+CD-*

13) Deque

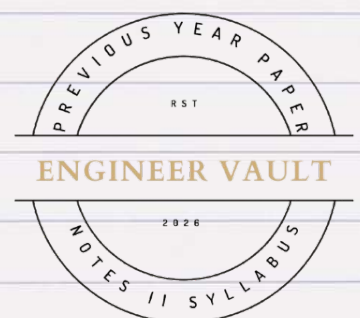
- Deque means in this both insertion & deletion happens at both ends.

* Dig :- $\leftarrow$ both side reusable $\rightarrow$
[10 20 30]

* Types

1) Input Restricted

- Insertion happens at one end.
- Deletion happens at both end.



- 1) Deletion - Deletion happens at one end.
 - 2) Insertion - Insertion happens at both end.
- * Application.
 - Use in sliding window
 - Use in Job scheduling.
 - * Eg:- Use in undo / redo operations.

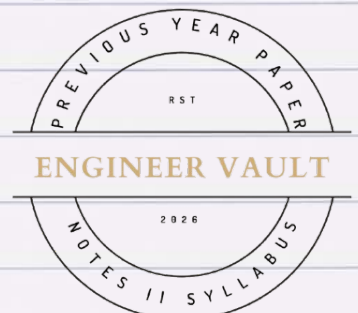
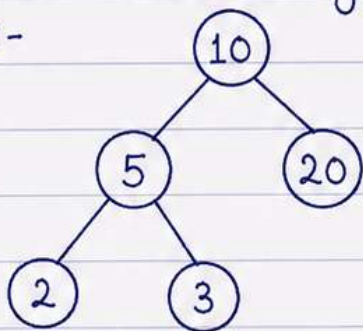
14] Doubly Linked List

- Doubly Linked List means in which each node contain data
prev
next.
- Doubly Linked List is linear data structure
- * Uses.
 - Easy Insertion & Deletion
 - Use in undo / redo operations.

15] Binary Search Tree

- Binary Search tree is a hierarchical data structure in which each node has 2 children left child & right child.
- If the value is smaller move to left.
- If the value is greater move to right.

Eg:-



16] Collision in hashing.

- Hash table have the actual value of fixed size & no. of keys are infinite.
- Because of this sometimes the hash table have the two same value with two diff. keys.
- This overlaps is called collision.
- We can not store the same value in the exact same cell so we need collision resolution technique.

17] Stack

- Stack is a linear data str.
- Stack follows LIFO
- Stack operations are push(), pop(), peek(), isEmpty(), isFull().
- In stack insertⁿ & deleteⁿ occurs at one end.
- Type of ADT

Queue

- Queue is a linear data str.
- Queue follows FIFO
- Queue operations are enqueue(), dequeue().
- In queue insertⁿ is done at rear & deleteⁿ is done at front.
- Type of ADT

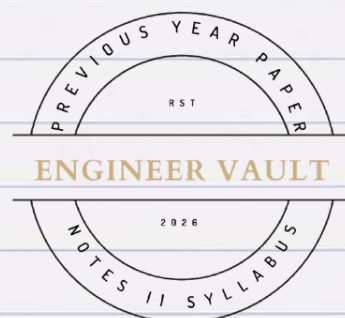
18] Array

- It is linear data structure
- It is a collection of similar data types
- It uses contiguous memory location.
- It is easy to access for indexing
- Indexing start from 0

Syntax:- data_type array_name[size];

Structure

- It is user defined which store the variables & diff. data types under the one name



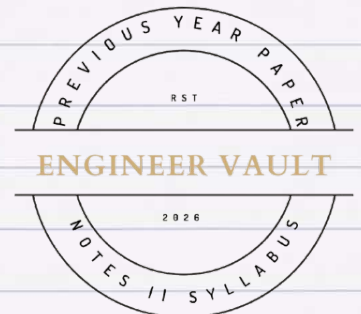
```
data-type member 1;  
data-type member 2;  
}
```

19] Asymptotic Notation

- It is a mathematical tool used to analyze the time complexity & space complexity.
- It is used to define the behaviours of the input size (n) increases.
- It is used to analyze best, worst, avg. time comp.
- Helps to compare the efficiency of the different algo.

* Types

- 1) Big Ω
- 2) Big O
- 3) Big Θ



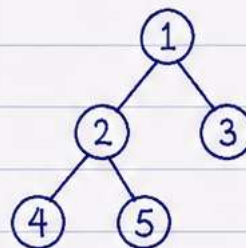
20] Preorder Traversal

- Preorder traversal visits a node

* Rule $\rightarrow$ Root $\rightarrow$ Left $\rightarrow$ Right

* Algo.

- o Visit a root
- o Traverse to left subtree
- o Traverse to right subtree



$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

* Uses

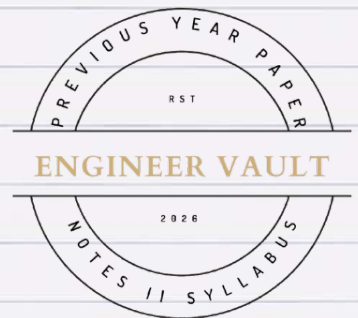
- Use to copy tree
- Use to get prefix expression.

21] Hash Table

- The index to determine where the value is stored in array called hash table.
- Provide efficient insertion & deletion.

Eg: $h(k) = \text{Key} \% 5$

No.	Calculate	Index
10	$10 \% 5$	0
20	$20 \% 5$	0
30	$30 \% 5$	0



The all no. have the same index (0).

22] Bubble Sort

- It is simple comparison based sorting algorithm
- It works by adding 2 adjacent no. and compare if they are in wrong order.

* T.C

- B.C = $O(n)$
- W.C, A.C = $O(n^2)$
- S.C = $O(1)$

Stable

In - place

* Ad → Fast accessing, Stable

* Dis →

23] Postfix expression

- In postfix, the operator comes after operands.
- Scan the elements from left to right
- Apply operands, push in stack
- Apply operands, pop 2 stack
- The final value is ready in stack

Eq → $23 * 54 +$

$$P = 2, P = 3, A^* = 6$$

$$P = 5, P = 4, A^* = 20$$

$$A^* = 20 + 6 = 26$$

24] Heap Sort

- Heap sort is the sorting algorithm in which & uses the binary heap.
- It works like selection sort, not faster.

* Time Complexity

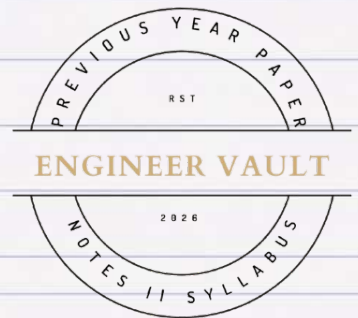
$$B.C, W.C, A.C = O(n \log n)$$

$$S.C = O(1)$$

Slower than quicksort

Not Stable

In-Place



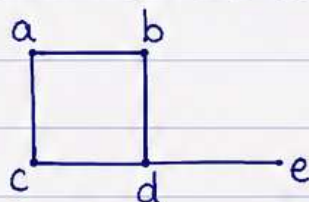
25] Application of stack

- It is used for expression evaluation
- Used in undo / redo operations.
- It converts infix expression to postfix or prefix.
- Used in backtracking

26] Graph

- A graph is a non-linear data structure
- It consists of vertices & edges
- Vertices represent node
- Edge represent connections
- Graph can be undirected or directed.

Eg:-



$$\text{Vertices} = \{a, b, c, d, e\}$$

$$\text{Edges} = \{ab, bd, ac, cd, de\}$$

27) Data structure

- Data structure is a way of organizing & storing data so that it can be used efficiently.

Eg:-

Array → It is a collection of similar data type.

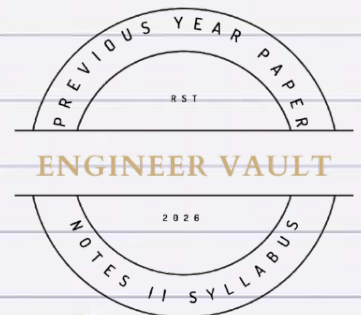
ex → $A = [10, 20, 30, 40]$

Stack → It follows LIFO

ex → Stack $[10, 20, 30]$

push 40 → $[10, 20, 30, 40]$

pop 30 → $[10, 20, 40]$



28) Dynamic Memory Allocation

- It means giving memory to a program while running.
- It prevents memory waste.
- Use limited of memory
- func() like malloc(), calloc() & free()

29) Pointers

- A pointer is a variable that stores the add. of another variable.
 - It allows direct access to memory locations.
 - Pointers are declared using * operator.
 - It support DMA.
 - Help in accessing elements of L.L & Trees.
- * Syntax = data type * var_name;

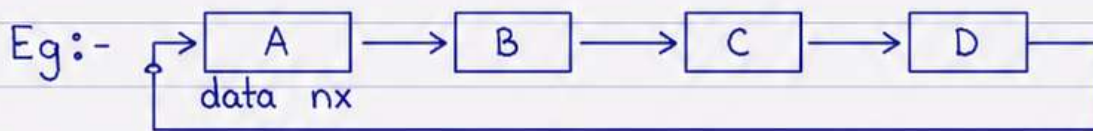
30) Header Linked List

- A header linked list contains a special node at the beginning.
- This node is called header node.
- Header node does not store actual value.

- It stores info like size or pointer
- It always points to 1st node
- It is a type of Linklist

31] Circular Linked List.

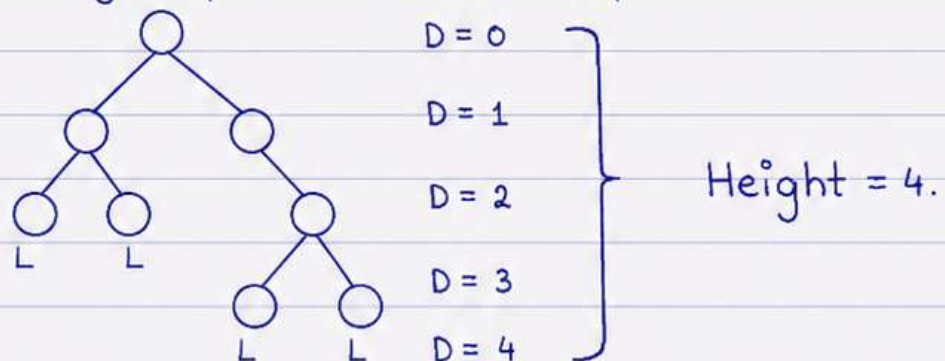
- C.L.L is a linked list in which the last node points to the first node.
- No NULL pointer.
- Can be singly or doubly circular
- Traversal can start from any node.



32] Depth of a binary tree

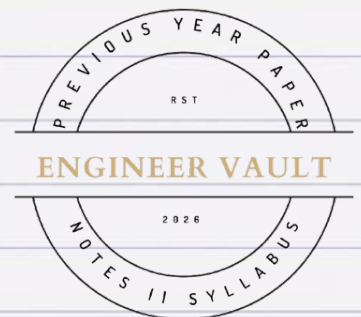
- It means no. of levels in a binary tree.
- It is measured from root to deepest leaf node
- Root node depth = 1
- Max. path length from root to leaf

Eg:-



33] Adjacency Matrix

- Used to represent a graph
- Graph is stored in 2D array
- If 1 edge then $\text{matrix}[i][j] = 1$
- If no edge then $\text{matrix}[i][j] = 0$
- Matrix size = $V \times V$ (V = no. of vertices)



Eg:- Vertices : A, B, C

Edges : $A \rightarrow B$, $B \rightarrow C$

	A	B	C
A	0	1	0
B	1	0	1
C	0	1	0

34] Warshall's Algorithm

- Used to find transitive closure graph
- Works on adjacency matrix
- Uses dynamic programming
- Eg $\rightarrow A \rightarrow B \rightarrow C$

	Initial A.M				Final Matrix		
	A	B	C		A	B	C
A	0	1	0	A	0	1	1
B	1	0	1	B	1	0	1
C	0	1	0	C	0	0	0

35] Dijkstra's Algorithm

- Used to find the shortest path.
- Single source $\rightarrow$ all vertices.
- Works on weighted graph
- No negative weight edge
- Use greedy approach.
- T.C = $O(n^2)$.

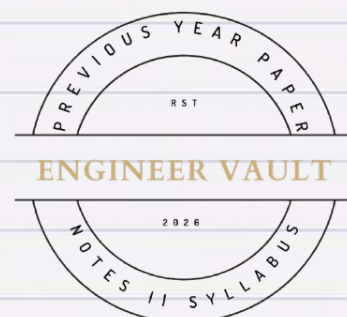
36] Adjacency List.

- Used to represent a graph
- Implement using array + linked list.
- Uses less memory.
- Eg:- $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

$1 \rightarrow 0 \rightarrow 2$

$2 \rightarrow 0 \rightarrow 1$

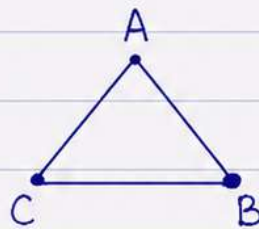
$3 \rightarrow 0$



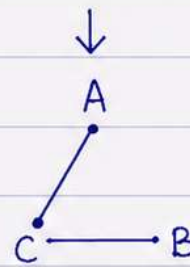
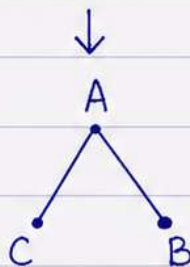
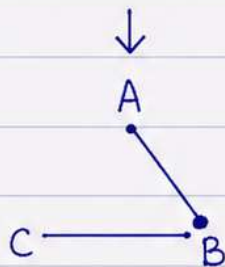
Spanning tree

- A spanning tree is a subgraph of a connected graph.
- It includes all vertices
- Contains no cycle
- Graph must be connected.

Eg → Graph



S.T



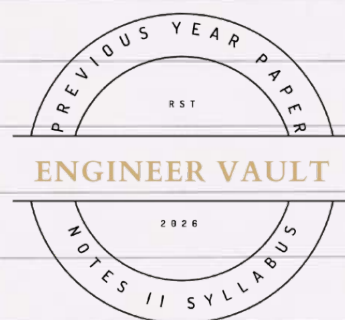
Set 'B' unit

Q2 → b]

- Conversion of infix to postfix expression using a stack- Stacks play a major role for evaluating and converting arithmetic expressions
- Stack is a linear data structure
- Stack follows the FIFO principle
- * Expressions are written in one of three forms :
 - Infix $(A + B)$
 - Prefix $+AB$
 - Postfix $AB+$

1] Evaluation of Postfix in Stack:-

- In postfix, the operator comes after operands.
- Scan the elements from left to right.
- Apply operands, push into stack.
- Apply operands, pop in 2 stack.
- final value is ready in stack.

Eg :- $23 * 54 +$ $P = 2 \quad P = 3, A^* = 6$ $P = 5, P = 4, A^* = 20$ $A^+ = 26$ 

2] Evaluation of Prefix in Stack:-

- In prefix, the operator comes before operands.
- Scan the elements from right to left.
- Apply operands, push into stack.
- Apply operands, pop in 2 stack.
- final value is ready in stack.

3] Evaluation of Infix in Stack.

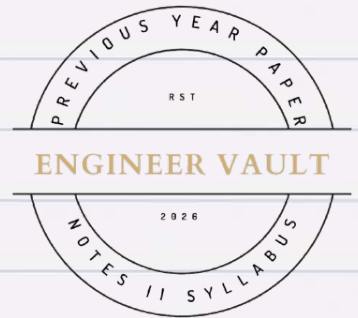
- In infix, the operator comes between operands.
- * Stack operations:- 1] Implementation 2] Solving the bracket

* Precedence

- * +, - → high
- * *, / → low

* Parenthesis :: It is normal handling.

Eg :- $(A+B) * C$.



* Conversion of infix to postfix expression

$(A+B) * (C-D)$

Size	Symbol	Stack operations	Link expressions
1	(	C	-
2	A	C	A
3	+	C+	A
4	B	C+	AB
5	)	-	AB+
6	*	*	AB+
7	(	* C	AB+
8	C	* C	AB+ C
9	-	* C-	AB+ C
10	D	* C-	AB+ CD-
11	)	*	AB+ CD-
12	End	-	AB+ CD- *

Set 2nd

Q5] → a]

Quick Sort

- Quick Sort is divide and conquer sorting method
- It works by choosing the pivot elements, partition array around array pivot elements, recursively sorting elements.

* Working -

- 1) Choose pivot element.
first, Second, last element.
- 2) Partition array around the pivot elements.

- Elements lower than the p.e goes to left
- Elements greater than the p.e goes to right

3) Recursively sorted subarrays

- Apply to the left subtree / subarray
- Apply to the right subtree / subarray

* Key characteristics

- Best case :- $O(n \log n)$
- Avg case :- $O(n \log n)$
- Worst case :- $O(n^2)$
- Simplicity :- $O(n \log n)$
- Not Stable
- In-place
- Faster than the merge sort.

* Eg :- Step 1 :- 10 80 30 90 40

↓

Step 2 :- 10 80 30 90 40

Swapping

:- 10 80 30 90 40

Step 3 :- 10 80 30 90 40

Step 4 :- 10 80 30 90 40

Sw 80 | 30

Step 5 :- 10 30 80 90 40

Step 6 :- 10 30 80 90 40

Sw 80 | 40

10 30 40 80 90

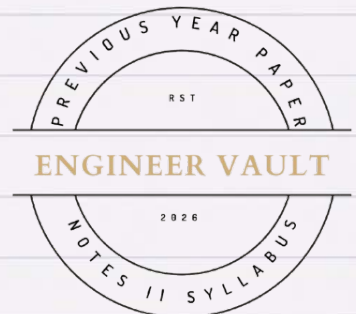
Final sorted array :- 10 30 40 80 90

* Advantages

- Faster accessing
- Faster than merge sort.

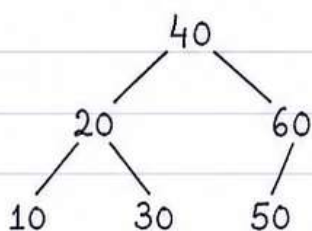
* Disadvantage

- Worst case
- Not Stable

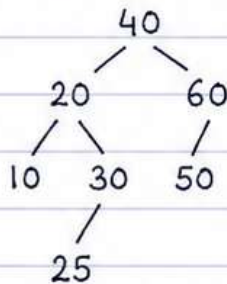


Binary Search Tree

- Binary Search Tree is a hierarchical data structure in which each node has left ^{2 children} node and right node.
- All the left subtree are smaller than root node.
- All the right subtree are greater than root node.
- * Insertion :- It is a process of adding a new node into BST.
- * Basic Concept of BST in insertion.
- Insertion always start from the root node.
- The value of insertion is to be compared with current node.
- If the value is smaller, traversal is done towards the left subtree.
- If the value is greater, traversal is done towards the right subtree.
- This process repeats until the NULL position is found.
- The new node is inserted at that position.
- * Algorithm
- 1. Create a new node with given value.
- 2. If BST is empty: Make a new value in root.
- 3. Else :- Compare the new value with root.
 - If the value is smaller, traversal is done towards the left subtree.
 - If the value is greater, traversal is done towards the right subtree.
- 4. Repeat the step 3 until the NULL position is found.
- 5. The new node is inserted at that position.
- * Eg :- Insert the elements :- 40, 20, 60, 10, 30, 50

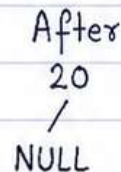
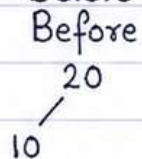


- * New insert 25
- 1) Start from root node
- 2) Go left or right compared the value.
- 3) Stop when empty place is found
- 4) Insert 25 there.
- * Updated BST after insertion of 25

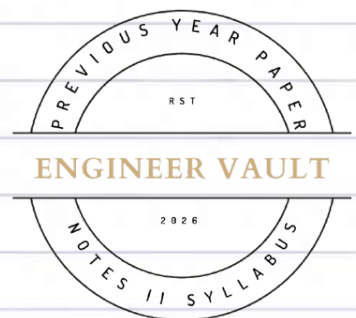
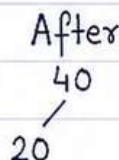
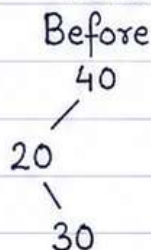


- * Deletion :- Deletion means removing a node from BST
- To delete a node from BST is complex due to maintain of properties.
- Three cases of Deletion :-
- Case 1 :- Node to be deleted a leaf node.
- No child node
- Delete a node
- Set as parent pointer as NULL

Eg :- Delete 10



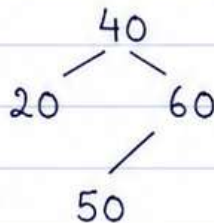
- Case 2 :- Node to be deleted with one child
- There is only child node left or right.
- Replace the node with child node
- Delete a node
- Eg :- Delete 30



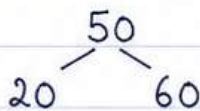
Case 2 :- Node to be deleted with two child

- Find inorder Successor
- Copy inorder Successor
- Replace inorder Successor

Eg :- Delete 40



- I.S = 50
- Replace 40 with 50
- Delete node 50



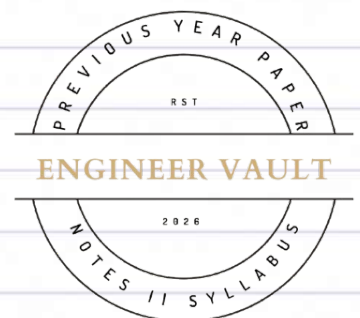
T.C	B.C	W.C
Insertion	$O(n \log n)$	$O(n)$
Deletion	$O(\log n)$	$O(n)$

* Algorithm

- 1) Root = NULL
Tree is empty, Root = Return
- 2) Key < Root
Set $\rightarrow$ left = deletebst.node
(root.left, key)
- 3) Key > Root
Set $\rightarrow$ right = deletebst.node
(root.right, key)
- 4) Case 1 :- No node (leaf node)
 - No child node
 - Delete node

Case 2 :- Node to delete with one child node

- Only one child node
- Replace node with child node



Delete a node delete

Case 3 :- Node to be delete with two child

- find Inorder Successor
- Copy Inorder Successor
- Replace Inorder Successor.

Set 2 q 5 b Merge Sort :- It divide and conquer the sorting algo. It divides the two halves, recursively sorted them and merge two halves back together.

* Working

- 1) Divides :- It divides the two halves until the array contain one element.
- 2) Conquere :- Recursively sorted the array.
- 3) Combined :- Combine the merge sort into the single sorted array.

* Key characteristic.

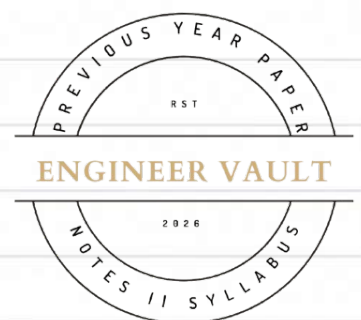
- Best Case - $O(n \log n)$
- Worst Case - $O(n \log n)$
- Avg Case - $O(n \log n)$
- Simplicity - $O(n^2)$
- Stable
- Not in place.

* Advantages

- Avg case performance
- Good performance
- Stable

* Disadvantages

- Not Stable In-place
- Slower than Merge Sort



* Comparison

Parameters	Merge Sort	Quick Sort
Techniques	Divide & Conquer	Divide & Conquer
Best Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n \log n)$	$O(n^2)$
Avg Case	$O(n \log n)$	$O(n \log n)$
Simplicity	$O(n^2)$	$O(n \log n)$
Stable	Yes	No
In-place	No	Yes

Set-1 q-3(g) * Binary tree :- It is a hierarchical data structure in which each node has 2 children left child and right child.

- To store the computer Memory there are 2 types

- 1) Array Representation.
- 2) Linked List Representation.

1) Array Representation

- Binary tree stored as array by mapping node to their array indices on their position.

- For stored array index 1

Left Child - $2 \times i$

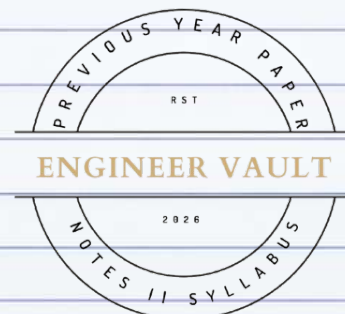
Right Child - $2 \times i + 1$

Parent - $i/2$

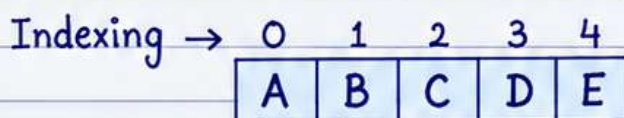
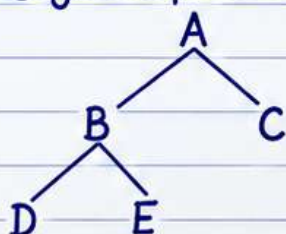
- For based index 0

Left Child - $2 \times i + 1$

Right Child - $2 \times i + 2$



* Array Representation



* Advantages

- Fast accessing
- Complete Binary Tree

* Disadvantages

- Not flexible
- Wastage of memory

2] Linked List Representation

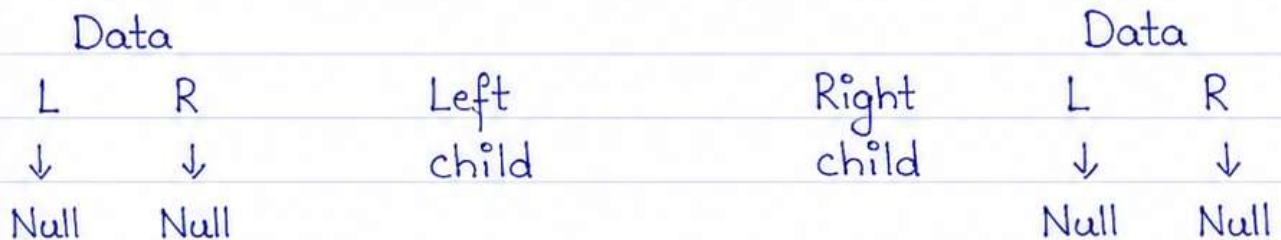
- Binary Tree have more representation on pointer.

* Node Representation

1) Data

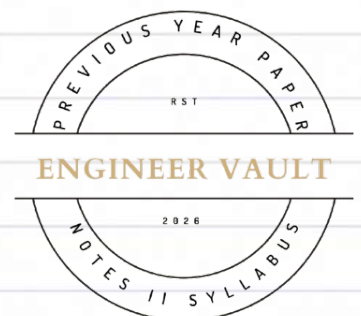
2) Pointer to the left child

3) Pointer to the right child



* Create / Declare Node

```
struct Node {
    int data;
    struct left Node;
    struct right Node;
}
```



* Advantages

- Fast accessing
- Easy to insert / delete

* Disadvantages

- Uses extra memory
- Slower accessing.

Tree Traversal Techniques each

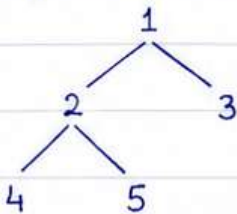
- Tree Traversal means visiting node of the tree at once in specific order.

Tree Traversal TechniquesDepth First Traversal

- 1) Preorder Traversal
- 2) Inorder Traversal
- 3) Postorder Traversal

Breadth First Traversal

Set 1 & 2
(q4 a) *

Binary Search Tree

Preorder = 1 → 2 → 3 → 4 → 5

Inorder = 4 → 2 → 5 → 1 → 3

Postorder = 4 → 5 → 2 → 3 → 1

* There are 3 types of DFT

1) Preordered Traversal

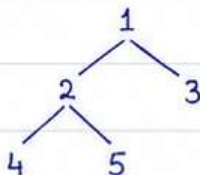
- Preordered Traversal visit a node in order.

* Rule → Root → Left → Right

* Algorithm

- Visit the root
- Traverse the left subtree
- Traverse the right subtree

* Eg :-



1 → 2 → 3 → 4 → 5

* Uses :- Use to copy tree
Use for the prefix expression

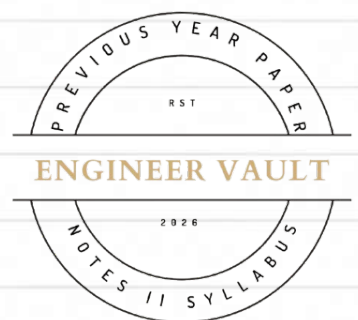
3) Inorder Traversal

- Inorder Traversal visit nodes in order.

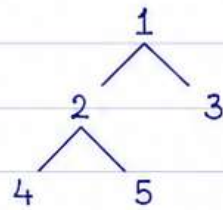
* Rule → Left → Root → Right

* Algorithm :- Traverse the left subtree

- Visits the root
- Traverse the right subtree



* Eg :



4 → 2 → 5 → 1 → 3

* Uses :- Use for the sorting in BST
Use for the infix expression

3) Postorder Traversal

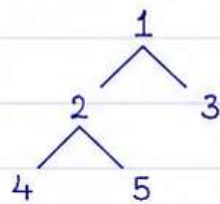
- Postorder Traversal visits a node in order.

* Rule → Left → Right → Root

* Algorithm

- Traverse left subtree
- Traverse right subtree
- Visits the root

* Eg :

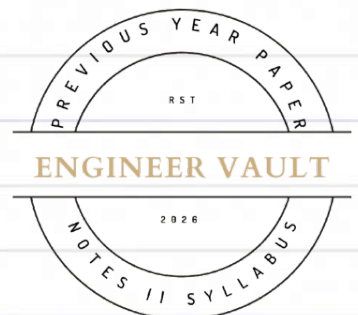
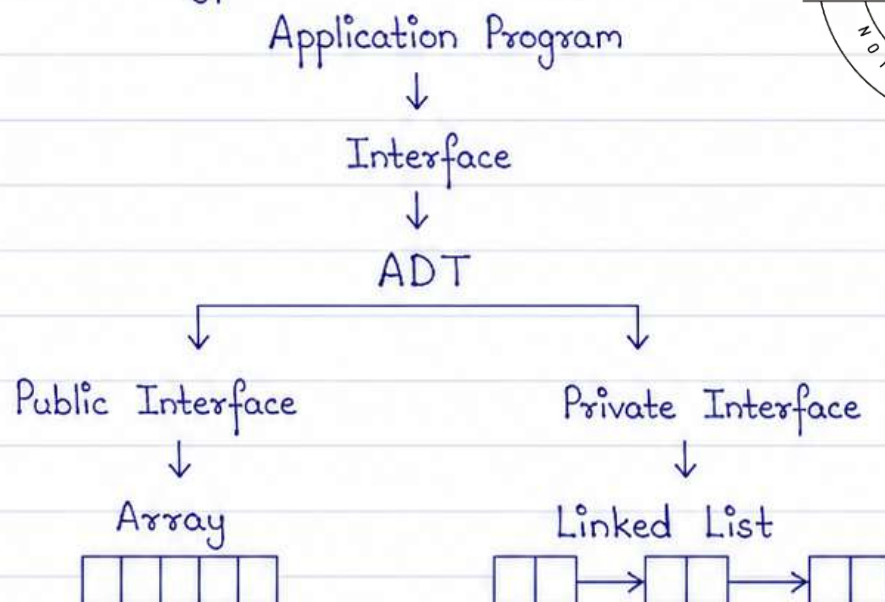


4 → 5 → 2 → 3 → 1

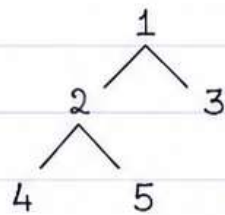
* Uses :- Use to delete a node
Use for the postfix expression

Set 2 Abstract Data Type

(Q 2a)



* Eg :



4 → 2 → 5 → 1 → 3

* Uses :- Use for the sorting in BST
Use for the infix expression

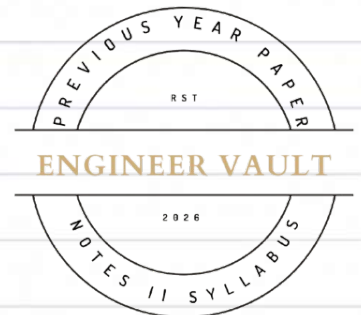
3) Postorder Traversal

- Postorder Traversal visits a node in order.

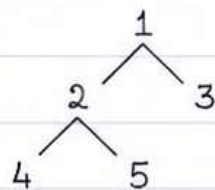
* Rule → Left → Right → Root

* Algorithm

- Traverse left subtree
- Traverse right subtree
- Visits the root



* Eg :

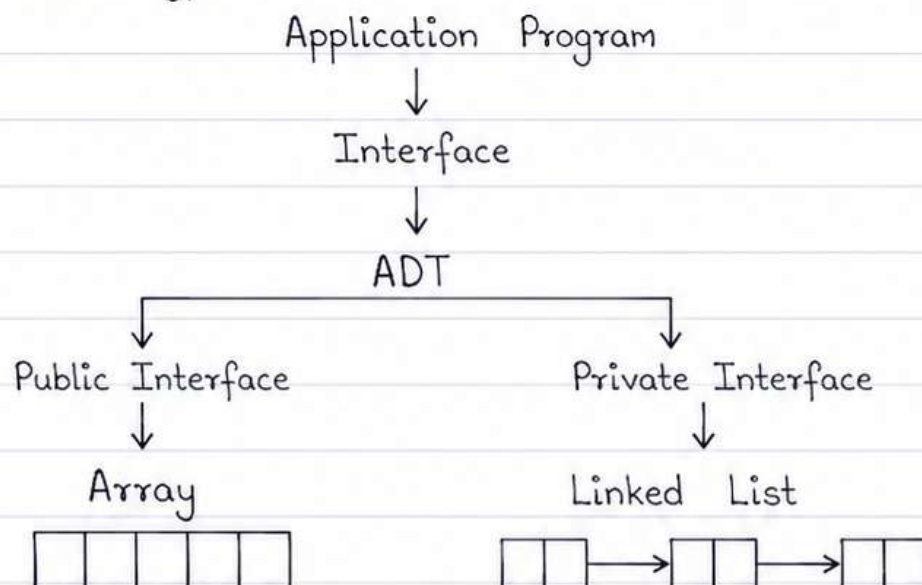


4 → 5 → 2 → 3 → 1

* Uses :- Use to delete a node
Use for the postfix expression

Set-2
(Q 2a)

Abstract Data Type



- ADT is an object which have the set of value & set of operations.
- ADT is logically makes the data structure.
- ADT have set of value and the set of operations.
- ADT is a data structure type makes implementation independtly.
- Tells that which operations should be perform and which should not performed.

* Uses of ADT.

- Helps in data management
- Use to define the data structure.
- Make program simple
- Improve program structure.

* Example:-

1) Stack

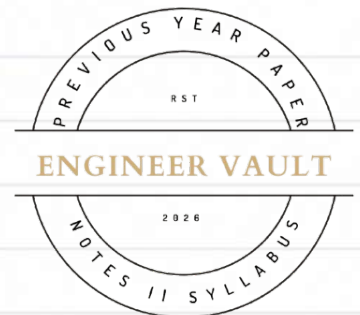
- Stack is a linear data structure.
- It follows the LIFO principle.
- In stack insertion and deletion is done at one end.
- Operations on stack
push(), pop(), isEmpty(), isFull().

- 1) push() → Insert the top element.
- 2) pop() → Remove the top element.
- 3) isEmpty() → Check is empty.
- 4) isFull() → Check is full.

2) Queue

- Queue is linear data structure.
- It follows the FIFO principle.
- Operations on queue → enqueue(), dequeue(), isEmpty(), isFull().

- 1) enqueue() → Insert element at end
- 2) dequeue() → Remove first element.



- 3) isEmpty() → Check is empty
4) peek() → Show front element.

2) Implementation

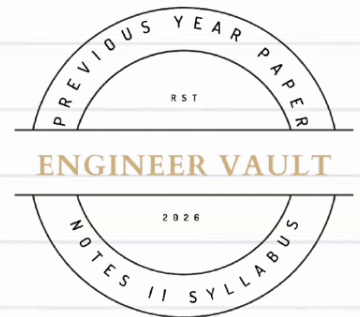
- i) The stack ADT implement the
- ii) Array
- iii) Linked list

* Advantages:-

- Easy to understand
- Provides data hiding

* Disadvantages:-

- Uses extra memory
- Takes more time



Set 3

Array Representation

(Q 2 a)

- Array Representation is a linear data structure
- Array is a collection of the similar data types.
- Array used the contiguous memory allocation.
- It is easy to accessing the indexing.

i) Contiguous Memory location.

- All the elements are stored in CML.
- It uses the i for the direct access of the formula.

* Formula

Formula for 1D

$$\text{Address}(i) = \text{BaseAdd} + (i * \text{Sizeof each element}).$$

* 1D Array

- Indexing value	0	1	2	3
Array Name	A	B	C	D

Let $\uparrow$ Array(i) =

$\frac{A}{\downarrow}$	$\frac{B}{\downarrow}$	$\frac{C}{\downarrow}$	$\frac{D}{\downarrow}$

Indexing

Data type

* 2D Array

- 2D Array can be stored as:-

i) Row Matrix order

- Entire 1st row is sorted, 2nd, so on.

ii) Column Matrix order

- Entire 1st column is sorted, 2nd, so on.

- Matrix

1	2	3
4	5	6

- Row Matrix order =

1	2	3	4	5	6
1	2	3	4	5	6

* Eg :- Step 1:-

10	20	30	40
----	----	----	----

pop 40

Step 2:-

10	20	30
----	----	----

push 35 for 3rd indexing

Step 3:-

10	20	30	35
----	----	----	----

push 40

Step 4:-

10	20	30	35	40
----	----	----	----	----

Final sorted array 10 20 30 35 40

* Advantages :-

1) Direct Access

It is fast access or direct access using the formula.

2) Easy to implement

It is easy to use and access.

3) Simple

It is use for sort, traversal, technique.

4) Indexing

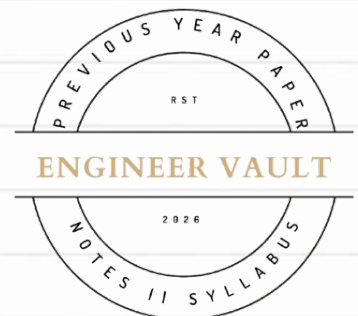
- Indexing start from 0.

Set 3 Queue

(Q3 a) - Queue is a linear data structure.

- Queue follows the FIFO principle.

- In this insertion is done at rear and deletion is done at front.

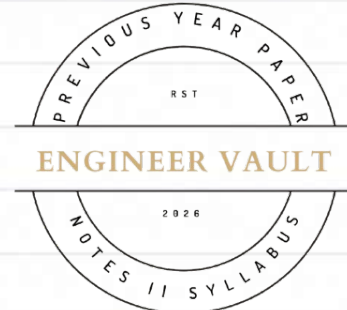


* Eg : 10 20 30 40 $\rightarrow$ Rear

* Types of Queue

- There are 4 types of Queue

- 1) Simple Queue
- 2) Circular Queue
- 3) Deque Queue
- 4) Priority Queue



- 1) Simple Queue - It is the basic form of queue where,
- Insertion - Enqueue is done at rear.
 - Deletion - Dequeue is done at front.

- * Features - Follows FIFO principle.

Uses linear array and linked list.

- * Limitations - When the element move forward then the empty space can't reused.

* Eg : - People standing in line of ticket counter.

* Applications :- CPU scheduling, printer queue.

* Dig = F - [10] [20] [30] $\rightarrow$ Rear

- 2) Circular Queue - It is the improved version of queue
- The last position is connected towards the first position.
 - Reduces the memory efficiency.

$$\text{Dig} = F - \overset{9}{[10]} \overset{9}{[20]} \overset{11}{[]} \overset{9}{[]} \overset{9}{[30]} \rightarrow R$$

* Features :- $\text{year} \ (\text{year}+1) \% \text{MAX}$

- No shifting of elements
- Efficient memory is used.

* Eg :- Circular buffer in memory

* Application :- Memory Management system, Real time system

- 3) Deque $\rightarrow$ It allows insertion and deletion at both ends.

* Dig = Both side reusable
 ← [10] [20] [30] →

* Types of Queue

- 1) Input Restricted Queue :- Insertion is done at one end.
• Deletion is done at both end.
- 2) Output Restricted Queue :- Deletion is done at one end.
• Insertion is done at both end.

* Features :- It supports both the stack as well as the queue.

* Eg :- Use for undo / redo operations.

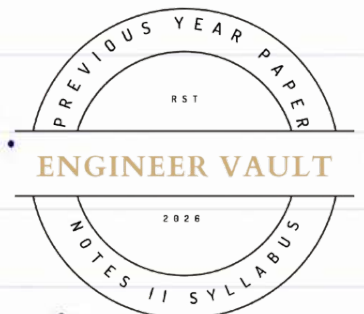
* Application :- Sliding window, Job scheduling

* Priority queue :- It is assigned to the priority.

- In priority queue it removes the elements with priority not FIFO order.

* Dig = 5 [highest], 3, 1 [lowest].

$[P = 5] \rightarrow [P = 3] \rightarrow [P = 1]$



* Features :- Higher priority removes the first

* Eg :- Use for the hospital emergency system.

* Types of priority queue :-

- 1) MAX priority queue $\rightarrow$ Higher element removes first.
- 2) MIN priority queue $\rightarrow$ lower element removes first.

* Applications :- CPU scheduling, Print scheduling.

* Bubble Sort

- It is simple to comparison based sorting algorithm.
- It works by comparing the two adjacent elements & swapping them if they are in wrong order.
- This process continues until the array is completely sorted.

* Working

- Step 1 - Create element of index $[i]$ & element of index $[i+1]$.
- Step 2 - If $A[i] > A[i+1]$, swap them.
- Step 3 - After the passes the largest element bubble ups to its correct position.
- Step 4 - This process repeat until no swap is required

* Key Characteristics

- 1) Sorting techniques : Comparison based.
- 2) Time Complexity $\rightarrow$ Best Case = $O(n)$
Avg/Worst Case = $O(n^2)$
- 3) Simple Complexity = $O(1)$
- 4) Stable
- 5) In-place.

* Eg :- 10 80 30 90 40

Step 1:- Placing 1st largest element to its correct position

$i = 0$ 10 80 30 90 40

 No swap

$i = 1$ 10 80 30 90 40

 Swap

$i = 2$ 10 30 80 90 40

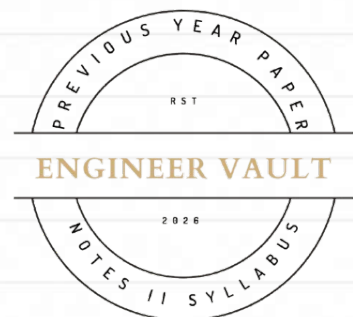
 No swap

$i = 3$ 10 30 80 90 40

 Swap

10 30 80 40 90

Step 2:- Placing 2nd largest element to its correct position



$i = 0$ 10 30 80 40 90

 No swap

i = 1 10 30 80 40 90

 No swap

i = 2 10 30 80 40 90

 Swap

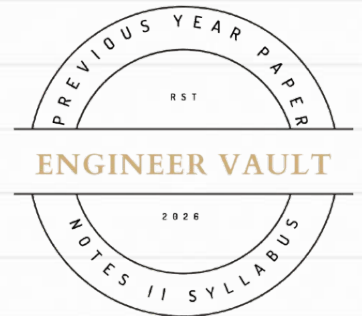
10 30 40 80 90

Step 3 :- Placing 3rd element to its correct position.

$i = 0$ 10 30 40 80 90

 No swap

← Sorted Element →
10 30 40 80 90



- * Advantage :- Easy to understand
 - It is stable
 - * Disadvantages :- Worst case & avg case complexity is $O(n^2)$
Slow for large data
 - * Selection Sort
 - Selection Sort is the simple based algorithm
 - It works by selecting the smallest element from the unsorted part of array.
 - * Working
 - 1) Divide the list into 2 parts :- Sorted (entire empty)
Unsorted (entire initially empty)
 - 2) From the unsorted array select minimum element.
 - 3) Repeat the number of element.
 - 4) Repeat the process until the array is sorted.
 - * Key characteristics
 - Sorting techniques = Comparison based
 - B.C, W.C, A.C $\Rightarrow O(n^2)$

* Simplicity = $O(1)$

* Not Stable

* In-place

* Eg :- 10 80 30 90 40

Step 1:- Unsorted arr[] = 10 80 30 90 40
 $\uparrow$
 current element.

Array becomes = 10 : 80 30 90 40
 Unsorted array Sorted Swap

Step 2:- 1st arr[] = 10 80 30 90 40
 $\uparrow$ $\uparrow$
 C.E m.E

Array becomes = 10 30 : 80 90 40
 Sorted S.W

Step 3:- Unsorted arr[] = 80 90 40
 array $\uparrow$ $\uparrow$
 C.E m.E

Array becomes = 10 30 40 : 90 80
 Sorted Swap

Step 4:- Unsorted arr[] = 90 80
 $\uparrow$ $\uparrow$
 C.E m.E

Array becomes = 10 30 40 80 90
 Sorted

* Advantages

- Easy to implement
- Simple to understand

* Disadvantages

- Not adaptive
- Time Complexity is always $O(n^2)$.

T.C B.S S.S

B.C $O(n)$ $O(n^2)$

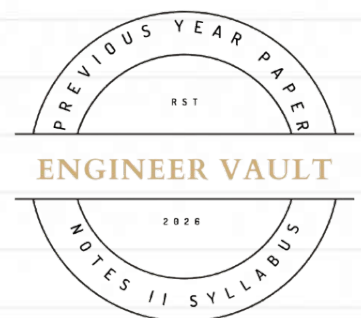
W.C $O(n^2)$ $O(n^2)$

A.C $O(n^2)$ $O(n^2)$

Simplicity $O(1)$ $O(1)$

Stable Yes No

In-place Yes Yes



* Radix Sort - It means the non comparative, digits based sorting algorithm.

- The no. of processing the least significant value to the most significant value.
- These sorting algorithm techniques.

* Working:

- The value of the max. no determine the no. of digits.
- Sorts no. digits to digits start from last digit.
- Repeat the number of digits.
- Use the stable sorting algorithm.

* Key characteristics

- Sorting techniques - Non-comparative.
- B.C, W.C, A.C $\Rightarrow O(nk)$
- S.C = $O(n+k)$
- Stable
- Not-in-place

* Eg:- 10 80 30 90 100

largest elements 100

So we do sort by :- Onils, Tens, Hundreds.

Step 1:- Sort by Units place

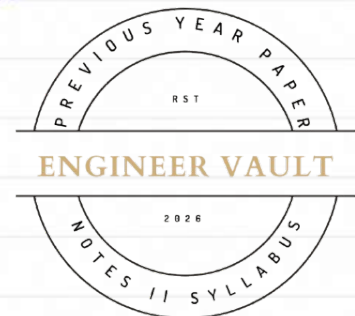
No.	Units digit
10	0
80	0
30	0
90	0
100	0

All units digits are 0, so order can't change.

Step 2:- Sort by Tens place

No.	Tens place
10	1
80	8
30	3
90	9
100	0

Result:- [100, 10, 30, 80, 90]



Step 3:- Sort by Hundreds place

No.	Hundreds digit
-----	----------------

10	0
----	---

80	0
----	---

30	0
----	---

90	0
----	---

100	1
-----	---

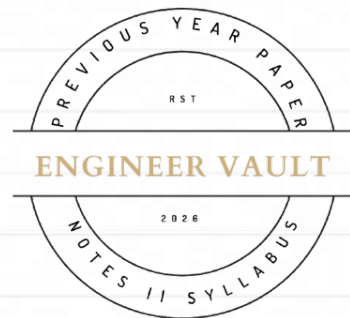
Result :- [10, 80, 30, 90, 100]

* Advantages

- It is In place
- Very fast for int & string

* Disadvantages

- Requires extra memory
- Sort must be stable.



* Heap Sort

- Heap Sort is a sorting algorithm in which it uses the binary heap.
- It works like selection sort, not faster.

* Working

- Pick up the largest or smallest element.
- Swap the last element to first element.
- Uses the heap to find the element efficiently.
- Repeats the process until the array is sorted.

* Key characteristic

Time Complexity

Best case = $O(n \log n)$

Worst case = $O(n \log n)$

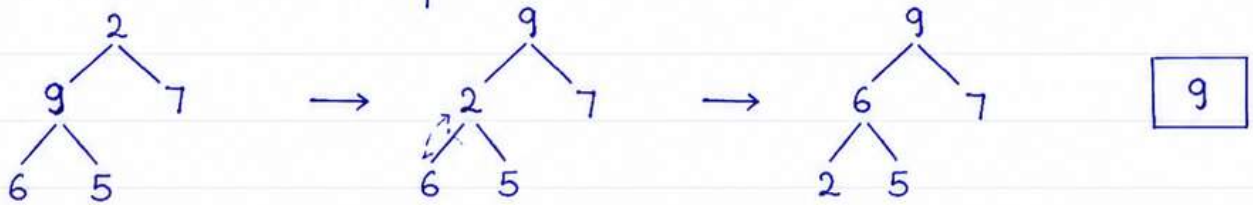
Avg case = $O(n \log n)$

Stable → No

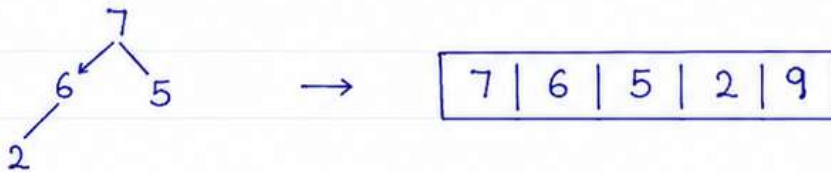
In - place.

* Eg :- 2 9 7 6 5

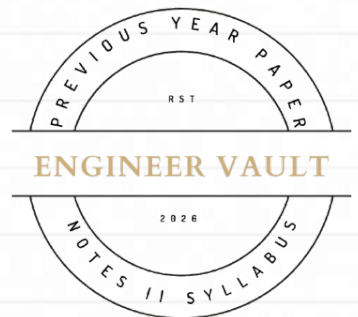
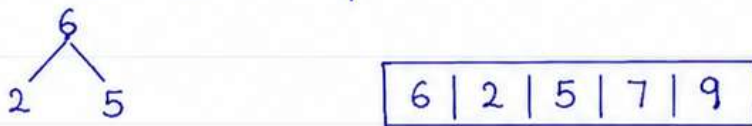
1) Make Max. heap



2) Swap the last element to first element



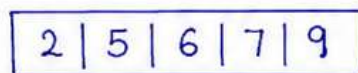
3) Reduce the heap



4) Use a heap to heapify



5) Repeat the process until the array is sorted



* Advantages

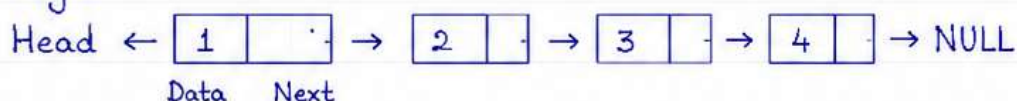
- Fast efficiency
- In-place

* Disadvantages

- Not Stable
- Slower than quicksort

- * Singly Linked List :- It is a collection of nodes where each node has :-
 data :- the actual value
 pointer(next) :- address of the next node

Eg :-



- Head point to the first node
- Each node has next node
- The last node is point to NULL.

* Algorithm of Singly Linked List.

* Insertion

1) Insertion at the beginning

- The new nodes becomes the first node of the list.

* Algorithm

- 1) Create a new node
- 2) Set newNode.next = head
- 3) Update head = newNode

Eg :- Before $\rightarrow$ Head $\rightarrow$ [10] $\rightarrow$ [20].

- Insert 5 at beginning.

After :- Head $\rightarrow$ [5] $\rightarrow$ [10] $\rightarrow$ [20].

2) Insertion at End

- The new node added to the last position.

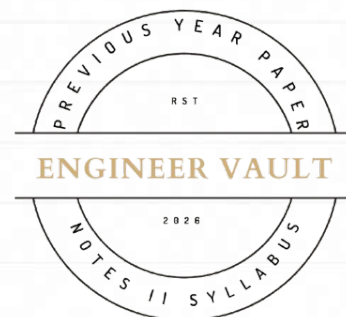
* Algorithm

- 1) Create a new node
- 2) Traverse till the last node
- 3) Set secondlast.next = newNode
- 4) Set newNode.next = NULL

Eg :- Before :- Head $\rightarrow$ [10] $\rightarrow$ [20].

Insert 30 at end.

After :- Head $\rightarrow$ [10] $\rightarrow$ [20] $\rightarrow$ [30].



3) Insertion at any specific position

- The new node is inserted at any specific position.

- Algorithm:- i) Traverse the node before position

ii) Set $\text{newNode.next} = \text{current.next}$

iii) Set $\text{current.next} = \text{newNode}$

* Eg :- Before :- Head $\rightarrow [10] \rightarrow [20] \rightarrow [30]$

Insert 25 after 20

After :- Head $\rightarrow [10] \rightarrow [20] \rightarrow [25] \rightarrow [30]$

* Deletion $\rightarrow$ Removing the node from list

1) Deletion at beginning

- The first node is removed

* Algorithm

1) Set $\text{head} = \text{head.next}$

2) Old first node is removed

* Eg :- Before :- Head $\rightarrow [10] \rightarrow [20] \rightarrow [30]$

Delete 10

After :- Head $\rightarrow [20] \rightarrow [30]$

2) Deletion at end

- The last node is removed

* Algorithm

1) Traverse the 2nd last node

2) Set $\text{secondlast.next} = \text{NULL}$

* Eg :- Before :- Head $\rightarrow [10] \rightarrow [20] \rightarrow [30]$

Delete 30

After :- Head $\rightarrow [10] \rightarrow [20]$

3) Deletion at specific position

- The new node is deleted at any specific position

* Algorithm

1) Traverse the node before the position.

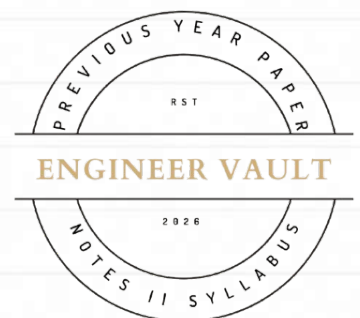
2) Set $\text{current.next} = \text{nodeToDelete.next}$

3) Delete the node.

* Eg :- Before $\rightarrow$ Head $\rightarrow [10] \rightarrow [20] \rightarrow [30]$

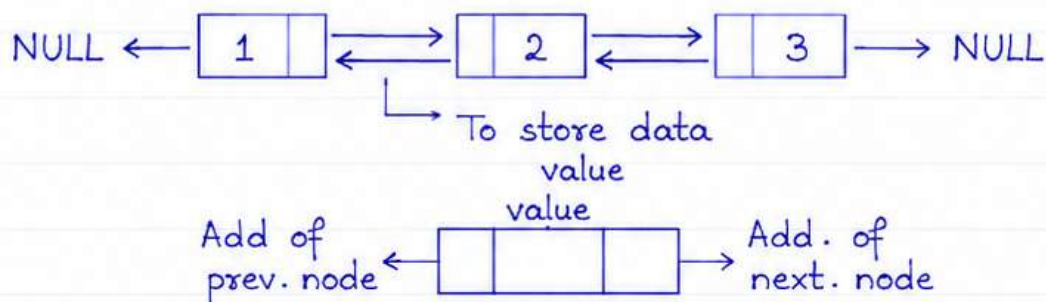
Delete 20

After $\rightarrow$ Head $\rightarrow [10] \rightarrow [30]$



* **Doubly Linked List** - It means it is a linear data structure in which each node has.

- 1) Data $\rightarrow$ the actual value
- 2) Previous Node $\rightarrow$ pointer of prev. node
- 3) Next Node $\rightarrow$ pointer of next node.



* **Operation**

1) **Insertion at beginning**

- The new node becomes the first element.
- Create a new node
- Set `newNode.next = head`
- Set `prev.node.next = NULL`
- Set `head.prev = newNode`
- Update `head = newNode`

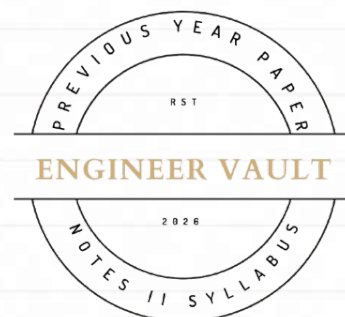
2) **Insert at end.**

* **Dijkstra's Algorithm**

- It is used to find the shortest path from the source node to a destination of a network.
- It works only on edges with non-negative
- It is a greedy algorithm.

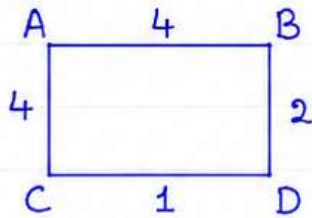
* **Working**

- It is used to find the shortest path of a source node.
- It works on non-negative edges.
- Start from source node from distance = 0.



* Example

- 1) Start with weighted graph



- 2) Assign dist. value

A = 0 (source node)

B = ∞ C = ∞ D = ∞

- 3) Pick the min. distance of vertex.

Pick A (dist = 0)

Update

B = 4

C = 4

- 4) Pick the next min. dist. of vertex.

Pick B (dist = 4)

Update

D = (4 + 2 = 6)

- 5) Pick the next min. dist. of vertex.

Pick C (dist = 4)

Update

D' = min(6, 4 + 1 = 5)

- 6) Mark all visited

A = 0

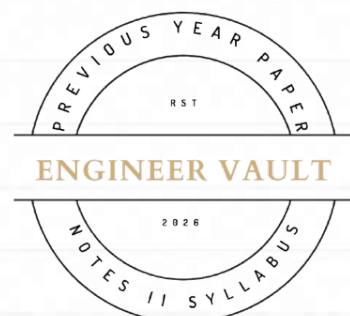
B = 4

C = 4

D = 5

* Uses.

- Uses for traffic navigation.



* Sequential Search

- Sequential Search means the element is checked one by one element.
- Sequential Search is also called as linear search
- Algorithm
- Start from the first element of the sorted array
- Compare the first element with the key
- If Match found, stop
- If not, Move to the next element of the list.
- Repeat the process until the element is found & list ends

* Examples: Given Array:- 10 80 30 90 40

Key=30 Step 1:- Compare the first element one by one.

Key=30 Not equal

	0	1	2	3	4
arr[] =	10	80	30	90	40
	C.E				

Step 2:- Compare the 2nd element of the list.

Key=30 Not equal

	0	1	2	3	4
arr[] =	10	80	30	90	40

The element is not equal to key, Move to next element.

Step 3:- Compare the 3rd element of the list.

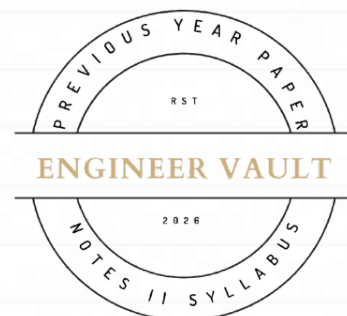
Key=30 Equal

	0	1	2	3	4
arr[] =	10	80	30	90	40

The element is equal to key, Match found

* Features

- Simple to implement
- Work as sorted & unsorted array
- Easy to understand



* Applications

- Use in searching the roll no.
- Use for the unsorted array

* Key characteristics

- Best case = $O(1)$
- Worst case = $O(n)$
- Avg case = $O(n)$
- Auxiliary Space = $O(1)$
- Space Complexity = $O(1)$

* Binary Search

- Binary Search means repeatedly divide sorted array into halves

* Algorithm:

- Find the middle element of the list.
- Compare the middle element with the key.
- If equal, stop.
- If it is smaller then go to left half
- If it is greater then go to right half
- Repeat the process until the element is found & list ends.

* Formula

$$\text{mid} = \frac{\text{low} + \text{high}}{2}$$

* Eg :- Given Array = 10 20 30 80 90

Step 1:- Sort the array

10	20	30	80	90
0	1	2	3	4

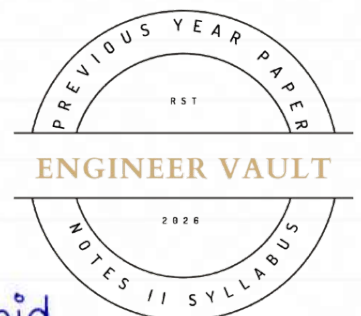
Key = 30

Step 2:- Set the initial value & calculate mid

10	20	30	80	90
0	1	2	3	4

Low = 0

High = 4



$$\text{mid} = \frac{0 + 4}{2} = \frac{4}{2} = 2 = A[\text{mid}] = A[2] = 30$$

The Middle element is 30

Step 3:- Compare the middle element with the key

Middle element = 30

Key = 30

Since both are equal, match found.

Divide the sorted array into halves.

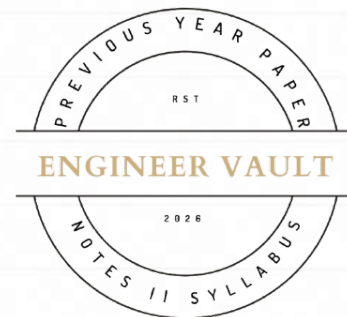
Easy to understand

* Applications

- Phone directories
- Dictionary word search

* Key characteristics

- Best Case - $O(1)$
- Worst Case - $O(n \log n)$
- Avg Case - $O(n \log n)$
- Auxiliary Space - $O(1)$
- Space Complexity - $O(1)$



* Hashing is a technique which identify the specific object form of the similar object

- It actuals converts the large arbitrary keys (like a name or email) into the small, fixed index
- This is done by using a hash function.
- Now, the index determined where the value should be stored in the array called hash table.

* Collision

- Hash table actually have the fixed sized & the possible no. of keys are infinite.

- Because of this sometimes the hash function have the same value of different keys.
- So this overlap is called as collision.
- Since we cannot store the two values in the same cell so we need collision resolution techniques

* Types of Hash functions.

- There are 5 types of hash functions.

1) Division Method

- Steps:- 1) Select the key
- 2) Apply the hash functions
- 3) Get the hash value.

Formula:- $h(k) = m \bmod k$

Eg:- Key = 50 , $m = 7$
 $50 \% 7 = 1$ index = 1

2) Multiplication Method

- Steps:- 1) Select the key
- 2) Apply the hash functions
- 3) Get the hash value

Formula:- $[m(KA \bmod 1)]$

Here A is always constant, the value is 0.681.

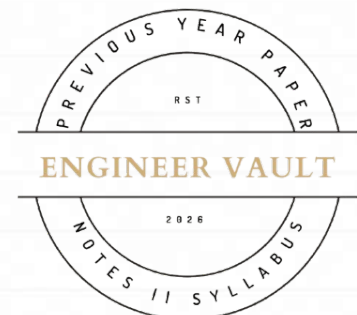
Eg:- Key = 20 , $A = 0.681$, $m = 10$
 $KA = (20 \times 0.681) = 12.36$
 $h(20) = (10 \times 0.36) = 3$
 index = 3

3) Mid-Square method

- Steps:- 1) Select the key
- 2) Apply the hash functions
- 3) Get the hash value

Formula:- $h(k) = \text{Middle digits of } k^2$

Key = 37
 $37^2 = 1369$
 index = 36



4) Folding Method

Steps:- Select the key

Apply the hash functions

Get the hash value.

Formula:- $h(k) = \sum (\text{parts of key})$

Key = 123456

Break = 12, 34, 56

$$= 12 + 34 + 56 = 102$$

$$h(k) = 12, m(10) = 2$$

$$\text{index} = 2$$

5) Digits Extraction Methods

Steps:- Select the key

Apply the hash functions

Get the hash value

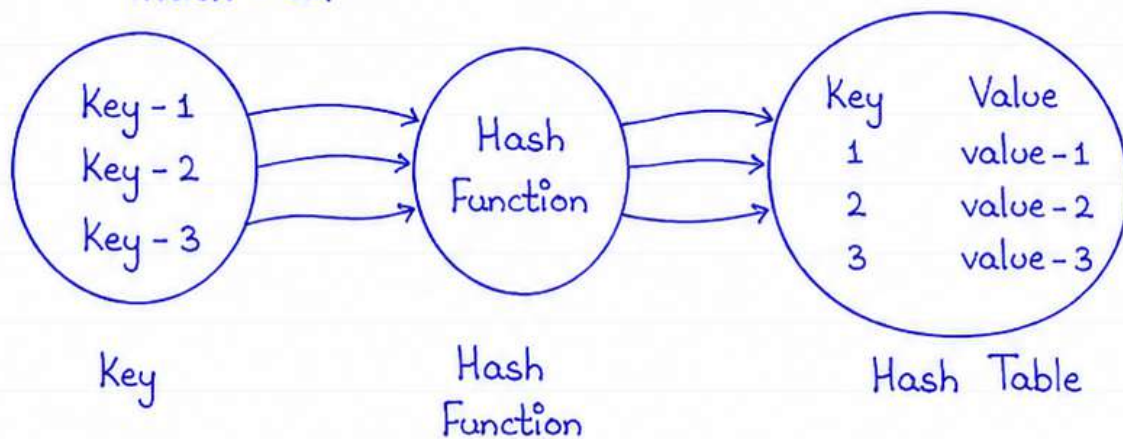
Formula:- $h(k) = \text{selected digits of key}$

Key = 123456

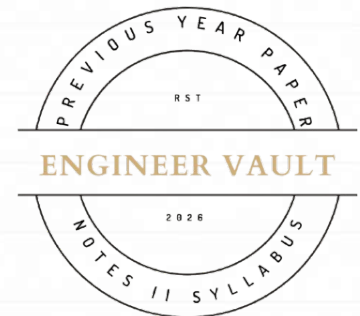
Select digits = 2nd & 4th

digits = 2, 4

$$\text{index} = 24$$



Components of Hashing



Collision in Hashing

Key = 10 $\rightarrow$ Hash function $h(10) = 0$
(for example)

Key = 15 $\rightarrow$ Key = 15 $h(15) = 0$

Hash table

Index	Value
0	10
1	
2	
3	

Collision Resolution Techniques

C R T

Separate Chaining
(Open Hashing)

Open Addressing
(Closed Hashing)

- 1) Linear Probing
- 2) Quadratic Probing
- 3) Double hashing

- Separate Chaining
- Separate Chaining is also called as open hashing.
- Separate chaining means each element of hash table is stored in linked list.
- If more than one element is found in hash table then it is stored in linked list.
- Collision handles the linked list together.
- It is easy to implement.
- It uses extra memory to store the linked list.
- Eg:- 10, 20, 30, 40, 50

$$\text{Key} = K \% 5$$

1) Create an empty hash table

Slot

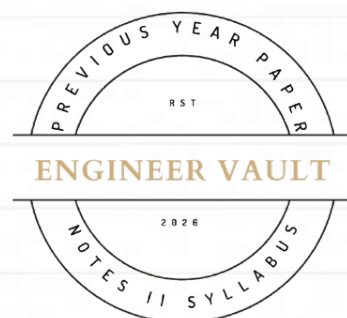
0

1

2

3

4



2) Calculate the hash value of the key

Slot

0

1

2 10

3

4

3) Insert the key of index

Slot

0

1

2 10 $\rightarrow$ 30

3

4

4) If collision occurs, move to the next element.

Slot

0 20

1

2 10 $\rightarrow$ 30

3

4

5) Repeat for all keys

Slot

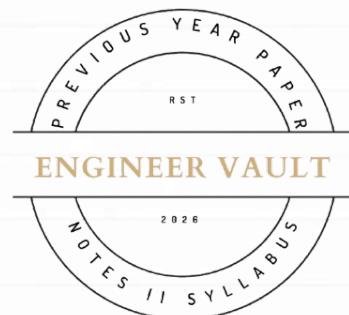
0 20 $\rightarrow$ 40

1

2 10 $\rightarrow$ 30

3

4



2) Open Addressing

- Open addressing is also called as closed hashing.
- Open addressing means store all elements inside hash table itself.
- If collision occurs, we search for an empty position.
- No linked list
- We check position one by one element

a) Linear Probing

- Linear Probing is an open addressing which is used to handle the collision.
- First find original hash index
- If collision occurs move to the next slot
- Continue until the free position is there.

Formula :- $h(k) = k \% 5$

Eg :- 10 20 30 40 50

1) Create an empty hash table

Slot

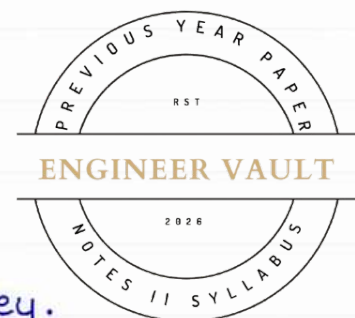
0

1

2

3

4



2) Calculate the hash value of the key.

Slot

0 10

1

2

3

4

2) Open Addressing

- Open addressing is also called as closed hashing.
- Open addressing means store all elements inside hash table itself
- If collision occurs, we search for an empty position.
- No linked list
- We check position one by one element.

a) Linear Probing

- Linear Probing is an open addressing which is used to handle the collision.
- First find original hash index
- If collision occurs move to the next slot.
- Continue until the free position is there.
- Formula :- $h(k) = k \% 5$
- Eg :- 10 20 30 40 50

1) Create an empty hash table

Slot

0

1

2

3

4

2) Calculate the hash value of the key.

Slot

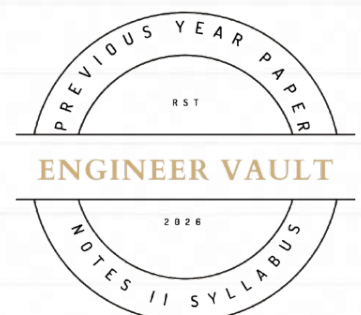
0 10

1

2

3

4



3) Insert the key of index

Slot

0 10

1 20

2

3

4

4) If collision occurs, move to the next element

Slot

0 10

1 20

2 30

3

4

5) Continue until free position gets.

Slot

0 10

1 20

2 30

3 40

4

6) Repeat for all keys

Slot

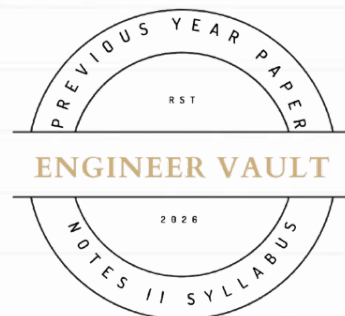
0 10

1 20

2 30

3 40

4 50.



* Quadratic Probing

- Quadratic Probing is an open addressing which use to handle the collision.
- First find the original hash index.

- If collision occurs, square the elements of the key.
- It reduces the clustering.
- Formula :- $h(k) + 1^2 + h(k) + 2^2 + \dots$

4) Double hashing

- To reduce clustering there are 2 hash functions.

* First hash function

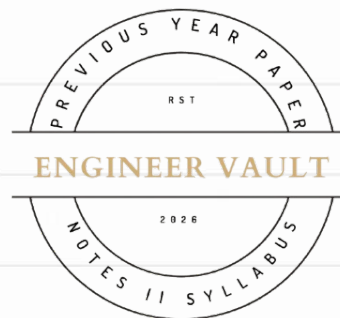
$$h_1(k)$$

Here it gives the initial value.

* Second hash function

$$h_2(k)$$

Here it finds the new element.



* Circular Queue

- It is same as the simple queue but it connects the last element to the first position forming a circle.
- Variables = front, rear, size queue.

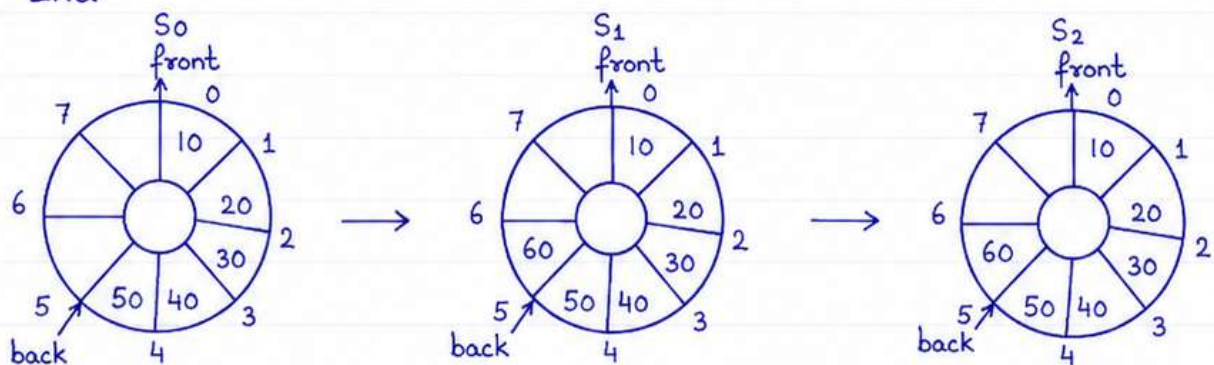
* Operations

1) Enqueue

- To add the element in queue

* Algorithm

- 1) If $((\text{rear} + 1) \% \text{size} == \text{front})$
Print "Queue Overflow"
- 2) Else $\text{rear} = (\text{rear} + 1) \% \text{size}$
 $\text{queue}[\text{rear}] = \text{value}$
- 3) If $(\text{front} == -1)$
 $\text{front} = 0$
- 4) End

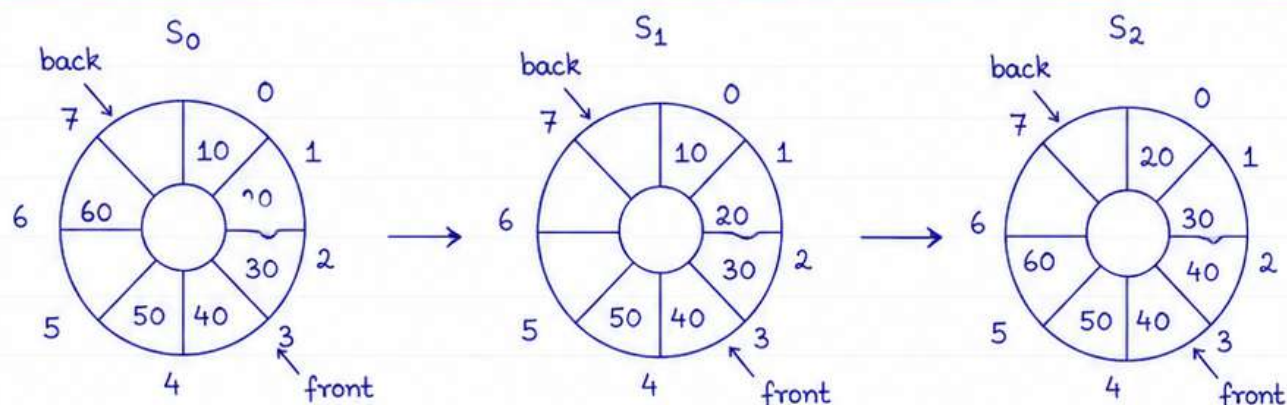


2) Dequeue

- To remove the element from queue

* Algorithm

- 1) If $(\text{front} == -1)$
Print "Queue Underflow"
- 2) Else
 $\text{value} = \text{queue}[\text{front}]$
- 3) If $((\text{front} + 1) \% \text{size} == \text{rear} + 1)$
- 4) End



Eg.

Initial value

front = -1 & rear = -1

After Enqueue

Q[0]	Q[1]	Q[2]	Q[3]
10	20	30	40

After dequeue

Q[]	Q[0]	Q[1]	Q[2]
	20	30	40

Empty Queue

Indexing	0	1	2	3	4
Queue	-	-	-	-	-

front = -1 & Rear = -1

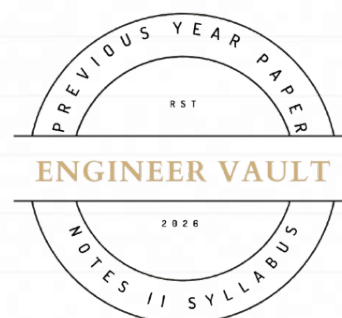
After Enqueue	Q[0]	Q[1]	Q[2]
	10	20	30

Indexing	0	1	2	3	4
Queue	[10	20	30	-	-]

After dequeue 10

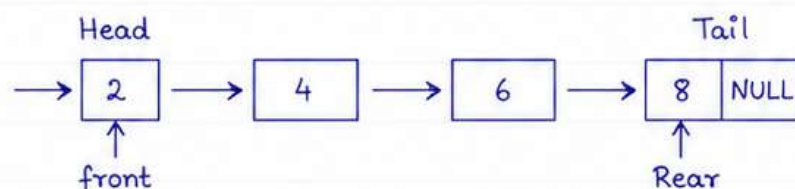
Indexing	0	1	2	3	4
Queue	[-	20	30	-	-]

front = -1 & rear = 2



* "Linked list implementation" of Queue

- Queue is a Linear Data Structure
- Queue follows the FIFO principle
- The first element is insert is first element is removed



- * Declaration of Queue
- Implementation of queue using L.L
- The node that contains
 - data :- the actual value
 - pointer :- address of next node.

* Two Pointer

- 1) Front
 - Head of queue
 - Point to 1st node
- 2) Rear
 - Tail of queue
 - Point to last node

* Enqueue

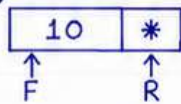
- To add the element in queue.

* Algorithm

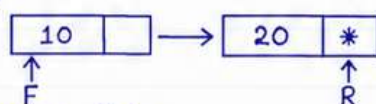
- 1) Create a new node with given value.
- 2) If queue is empty
 - front = Rear = new Node
- 3) Else
 - Rear → Next = new Node

* Eg:-

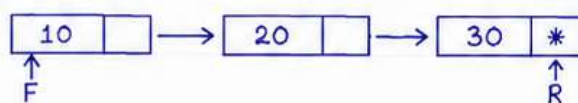
Enqueue 10



Enqueue 20



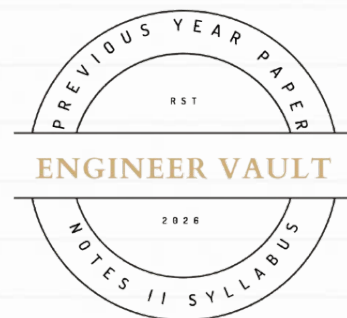
Enqueue 30



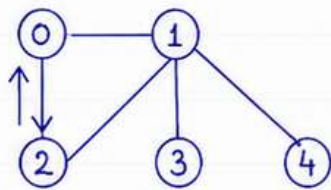
* Dequeue.

- To remove the element from queue

* Algo.



Step 3 :- We visit neighbour 0 : i.e 1



vis[]

T	T	F	F	F
---	---	---	---	---

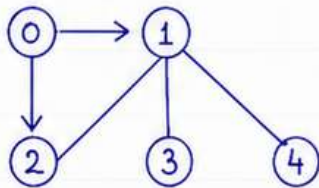
res[]

0	1	-	-	-
---	---	---	---	---

dfsRec(1)

dfsRec(0)

Step 4 :- We visit neighbour 1 : i.e 2



vis[]

T	T	T	F	F
---	---	---	---	---

res[]

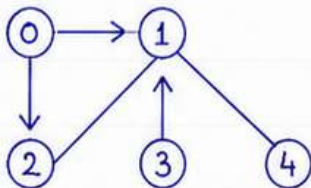
0	1	2	-	-
---	---	---	---	---

dfsRec(2)

dfsRec(1)

dfsRec(0)

Step 5 :- We visit neighbour 2 : i.e 3



vis[]

T	T	T	F	F
---	---	---	---	---

res[]

0	1	2	3	-
---	---	---	---	---

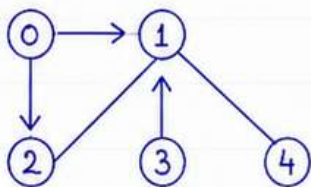
dfsRec(3)

dfsRec(2)

dfsRec(1)

dfsRec(0)

Step 6 :- Back tracking from 3



vis[]

T	T	T	F	F
---	---	---	---	---

res[]

0	1	2	3	-
---	---	---	---	---

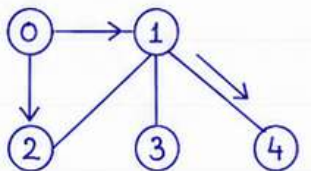
dfsRec(1)

dfsRec(2)

dfsRec(1)

dfsRec(0)

Step 7 :- We visit neighbour 1 : i.e 4



vis[]

T	T	T	T	T
---	---	---	---	---

res[]

0	1	2	3	4
---	---	---	---	---

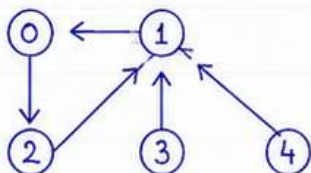
dfsRec(4)

dfsRec(2)

dfsRec(1)

dfsRec(0)

Step 8 :- All nodes are visited



vis[]

T	T	T	T	T
---	---	---	---	---

res[]

0	1	2	3	4
---	---	---	---	---

(stack empty)

Stack is
empty

DFS order = 0 1 2 3 4.

* Prim's Algorithm

- Prim's Algorithm is a greedy algorithm which is used to find the minimum spanning trees.
- Starts from the vertex
- It keeps adding the minimum weighted graph
- It connects all edges with the minimum total cost.
- a node inside the tree
- to a node outside the tree
- No cycle is formed

* Algorithm:

- Start from any one vertex (0)
- Put the vertex in MST set
- To find all edges connecting to MST vertices to outside vertices.
- Choose the minimum weight edge.
- Add the edge and vertex to the MST
- Repeat step 3 & 5 until the vertex are found

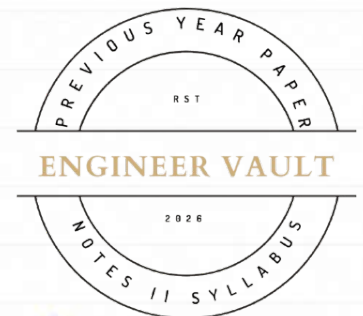
* Exist

* Eg



* Uses

- Use for the networks
- Use for the min. spanning trees.
- Use to find a min. spanning trees.
- Use to make design the model
- Use to make the pipeline.
- Use for the computer networks



* Advantages

- It is use to find minimum cost spanning tree.
- Easy to understand
- Simple to implement

* Disadvantages

- It works only in connected graphs
- Needs extra memory
- Slow for large sets.

* Time Complexity

- Best case $O(E+V) \rightarrow$ using Binary Heap
- Worst case $O(E(\log V \log V)) \rightarrow$ using Fibonacci Heap

* Space Complexity

$$O(V+E)$$

↓ ← no. of edges
vertex

* Kruskal's Algorithm

* Minimum Spanning Tree

The Minimum spanning tree is :-

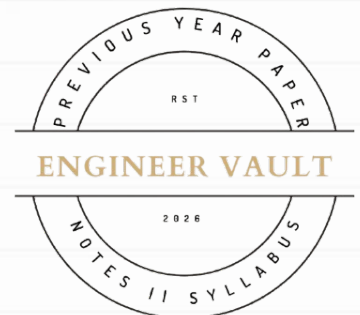
- Connects all the vertices
- Has no cycles
- Has minimum cost spanning tree.

* Kruskal's Algorithm

- Kruskal's Algorithm is a greedy algorithm in which it is used to find the MST connected, weighted, undirected.
- It works by selecting the edge of the smallest vertex and has no cycle form.

* Algorithm

- Select edge in the increasing weight order.
- Pick the edge of smallest weight.
- If adding weight has no cycles put in MST.
- If the cycle forms, discards the edge.



- Repeat this process until the MST(V-1) edges.

* Uses

- Use for networks
- Use for making roads
- Use for designing
- Use to connect nodes with minimum cost.

* Advantage

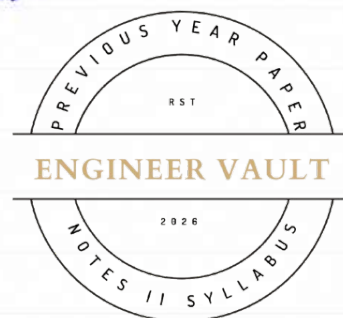
- Easy to understand
- Simple to implement
- Has the minimum cost spanning trees.

* Disadvantage

- Takes more time for sorting
- Needs extra memory
- Works for connected graphs.

* Time Complexity

- Best Case = $O(E \log E)$
- Worst Case = $O(E \log E)$
- Avg Case = $O(E \log E)$
- Space Complexity = $O(V)$.



* Asymptotic Notation

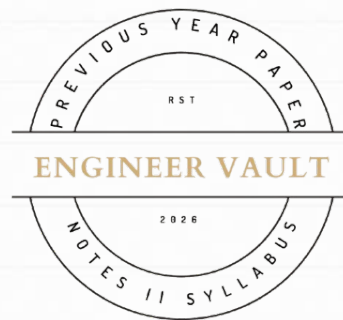
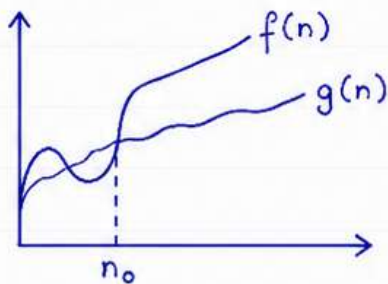
- It is the mathematical tool used to analyse the time complexity of an algorithm & space complexity
- It is used for analyze the best, worst & avg case of time complexity
- It is used to define the behaviour of input size (n) increases.
- Help to compare the efficiency of different algorithm.

* Types of Asymptotic Notation

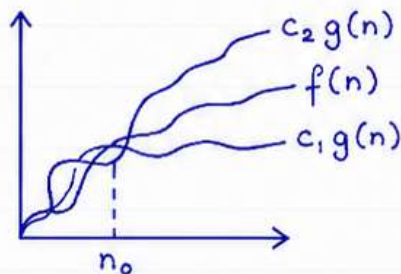
- 1) Big Ω notation $\rightarrow$ Best Case
- 2) Big Θ notation $\rightarrow$ Avg Case
- 3) Big O notation $\rightarrow$ Worst Case

1) Big Ω Notation

- Represent as the best case of complexity.
- Gives lower bound of an algorithm.
- It is used in best case situation.
- Shows the minimum growth rate of an algorithm.
- Written as $\Omega(f(n))$
- Describes the shortest execution time

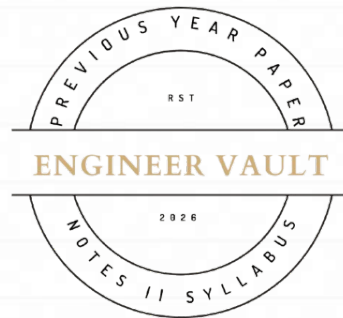
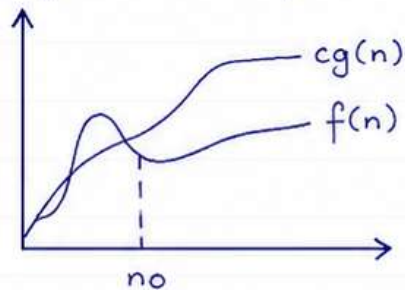
2) Big Θ Notation

- Represent as the avg. case of complexity.
- Gives upper and lower bound of an algorithm.
- It is used to analyze performance when best & worst case is same.
- Shows the exact growth rate of an algorithm.
- Written as $\Theta(f(n))$
- Helps to compare the large input size (n) algorithms.

3) Big O Notation

- Represent the worst case of complexity.
- Gives the upper bound of an algorithm.
- Shows the max. growth rate of algorithm.
- It is used to analyze the performance in

- In worst case situation
- Written as $O(f(n))$
- Helps to compare the large input size of algorithm



* Threaded binary Tree

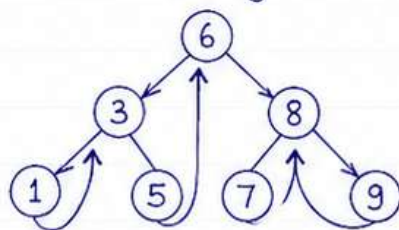
- Threaded binary tree is a binary tree data structure the NULL right or left pointer replace by the thread.
- The thread is the inorder successor & the inorder predecessor.
- Right pointer is inorder successor
- Left pointer is inorder predecessor.

* Types of threaded binary tree.

1) Single TBT

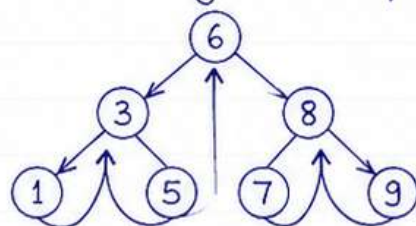
- It only uses the right NULL pointer.

Eg:-

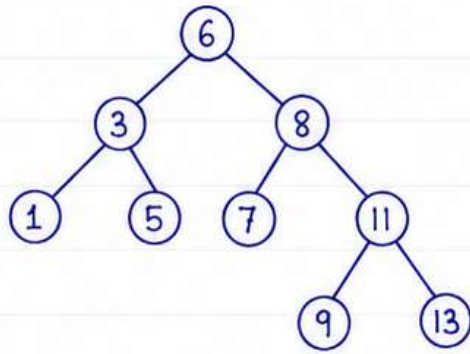


2) Double TBT

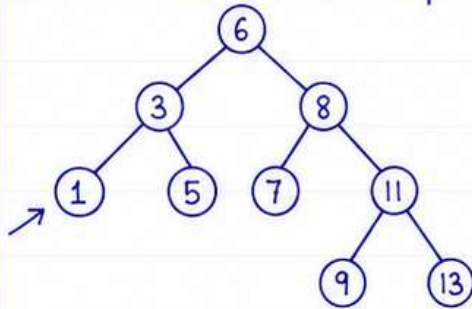
- It uses both right & left NULL pointer.



Eg:-

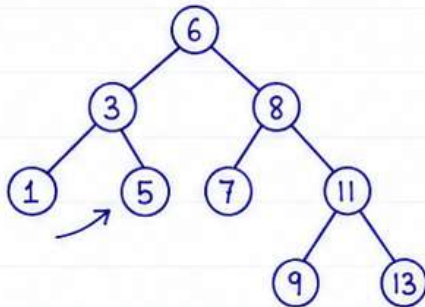


S1:- Start at leftmost node, print it



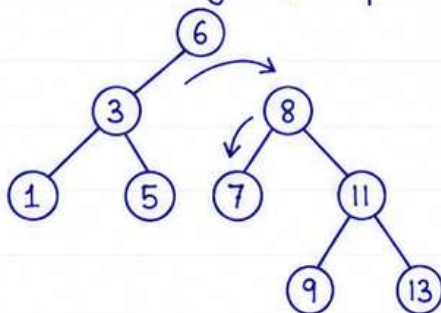
O/P
1

S2:- Follow thread to Right node, p.i



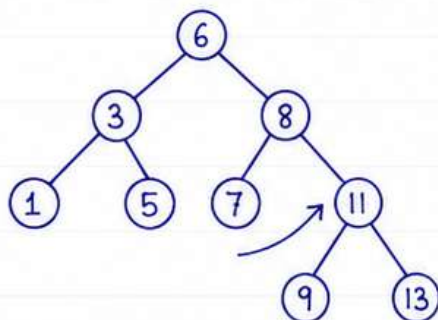
O/P
1
3

S3:- Go right, leftmost, p.i

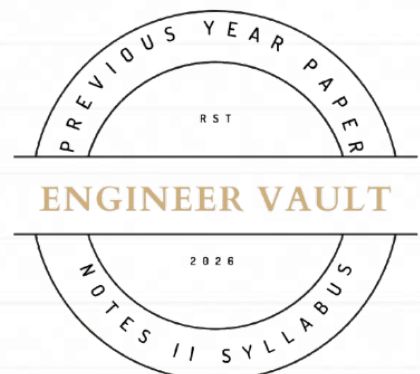


O/P
1
3
5

S4:- Follow thread to Right node, p.i



O/P
1
3
5
8



Singly Linked List	Doubly Linked List	Circular Linked List
1) Beginning (I) - Create a new node - Set <code>newNode.next = head</code> - Update <code>head = newNode</code>	1) Beginning (I) - Create a new node - Set <code>newNode.next = head</code> - Set <code>prevNode.next = NULL</code> - Set <code>head.prev = newNode</code> - Update <code>head = newNode</code>	1) Beginning (I) - Create a new node - Set <code>newNode.next = head</code> - Traverse to the last node - Set <code>lastNode.next = newNode</code> - Update <code>head = newNode</code>
2) End - Create a new node - Traverse till last node - Set <code>lastNode.next = newNode</code>	2) End - Create a new node - Traverse till last node - Set <code>lastNode.next = newNode</code> - Set <code>newNode.prev = lastNode</code> - Set <code>newNode.next = NULL</code>	2) End - Create a new node - Traverse till last node - Set <code>lastNode.next = newNode</code> - Set <code>newNode.next = head</code>
3) Specific Position - Traverse the node before the position - Set <code>newNode.next = current.next</code> - Set <code>current.next = newNode</code>	3) Specific Position - Traverse the node before the given position. - Set <code>newNode.next = current.next</code> - Set <code>newNode.prev = current</code> - Set <code>current.next.prev = newNode</code> - Set <code>current.next = newNode</code>	3) S.P - Traverse the node before the position - Set <code>newNode.next = current.next</code> - Set <code>current.next = newNode</code>
1) Beginning (D) - Set <code>head = head.next</code> - Old first node is removed	1) Beginning (D) - Update <code>head = head.next</code> - Set <code>head.prev = NULL</code> - Old first node is delete	1) Beginning (D) - Traverse to the last node - Set <code>head = head.next</code> - Set <code>lastNode.next = head</code> - Delete old first node
2) End - Traverse till last node - Set <code>secondLast.next = NULL</code> - Delete the node	2) End - Traverse till 2nd last node - Set <code>secondLast.next = NULL</code> - Old last node is delete	2) End - Traverse to the 2nd last node - Set <code>secondLast.next = head</code> - Delete last node
3) Specific position - Traverse the node before the position - Set <code>current.next = node to delete.next</code> - Delete the node	3) Specific Position - Traverse the node before the given position - Set <code>node.prev.next = node.next</code> - Set <code>node.next.prev = node.prev</code> - Delete the node.	3) Specific Position - Traverse to the node before the position. - Set <code>current.next = nodeToDelete.next</code> - Delete the node.